

VarBlockSpMM: Shape-Aware CUDA Execution for Packed Variable-Block Sparse-Dense Products

Cohen Brunner Thomas
Independent Research Project

6 September 2026

Abstract

VarBlockSpMM computes FP32 sparse-dense products directly from dense blocks with independently variable row and column partitions. A row-owned CUDA kernel specializes dense panel width, reuses sparse values through multiple accumulators, and selectively stages and double-buffers dense input tiles. RHS 8 uses eight-row tiles and two accumulators per thread for grids with at least 512 block rows, with a scalar fallback for smaller matrices. Its direct execution uses no output atomics or temporary workspace. We evaluate the optimized implementation on one NVIDIA GeForce RTX 5060 Ti using 128 synthetic configurations, a 40-configuration extension for matrix size and degree imbalance, and three published structural matrices with explicit partitions. Each configuration has three fresh process launches with randomized method order and retained raw timings. On the core synthetic grid, geometric speedups are 3.518 over a per-process lower envelope of four CSR configurations and 4.769 over cached grouped cuBLAS. Fixed-block BSR is faster in 23 of 32 uniform configurations. Compact CSR is faster in 12 of twelve application matrix and width configurations under the tested low-fill partitions. These controls separate the benefits of the execution policy from baseline setup overhead and block fill. The results describe performance on one device. Application relevance depends on the partitions and density of stored blocks.

1 Introduction

Sparse matrices often contain dense submatrices that are useful units of storage and computation. Fixed-size block sparse row storage exposes this structure when all blocks have the same dimensions. Variable partitions instead produce rectangular blocks of different heights and widths. Scalar expansion loses explicit block reuse, while padding every block to a maximum shape can increase storage and arithmetic.

We study $C = AB$, where $A \in \mathbb{R}^{M \times K}$ is a packed variable-block sparse row matrix and $B \in \mathbb{R}^{K \times N}$ and $C \in \mathbb{R}^{M \times N}$ are column-major dense panels. The supported block dimensions are 8, 16, ..., 64, with independently chosen row and column partitions. Dense panel widths are $N \in \{8, 16, 32, 64\}$. The implementation uses FP32 values and accumulation for non-transpose multiplication with $\alpha = 1$ and $\beta = 0$.

We ask when retaining dense variable blocks benefits a direct row-owned implementation, compared with persistent library baselines that reuse metadata and have suitable algorithm choices. We study a compact format, a width-specialized kernel family, and a dispatch policy chosen through measurement. The controls include fixed-block BSR for uniform blocks and compact scalar CSR for application partitions that introduce internal zeros.

Grouping a scalar sparse matrix into dense blocks can introduce many stored zeros. Naturally dense blocks avoid this penalty. Kernel speed alone therefore does not establish whether packing suits the source application.

2 Related work

Variable blocking is established prior art. Vuduc and Moon studied variable and unaligned blocks for sparse matrix-vector multiplication [6]. OSKI defines variable block row storage through independent logical row and column partitions [7]. VarBlockSpMM uses this structural model for a dense multi-column right operand on CUDA.

GPU SpMM work has examined row assignment, load balance, instruction-level parallelism, and reuse. Yang, Buluç, and Owens describe row-splitting and merge-based approaches [1]. GE-SpMM caches sparse rows and merges warps to reduce redundant loads [8]. Tensor-core methods such as Okanovic et al. trade packing and preprocessing for dense matrix-instruction throughput [2]. These techniques motivate the present kernel, but this study does not establish superiority over compatible research implementations.

The motivating application is adaptive construction of hierarchical H^2 matrices. Boukaram et al. describe a non-uniform batched BSR product implemented through non-uniform batched GEMM operations [5]. The present work addresses the underlying packed-block product. It does not benchmark a complete hierarchical construction pipeline or its native partitions.

NVIDIA provides scalar and fixed-block SpMM through cuSPARSE [3], and per-group GEMM dimensions through grouped cuBLAS [4]. These form the library controls. Asynchronous global-to-shared copies can overlap tile movement and arithmetic [9]. These mechanisms are established techniques. We combine and evaluate them for this particular variable-block operation.

3 Representation and kernel

Let block-row boundaries be $r_0 < \dots < r_R$, and block-column boundaries be $c_0 < \dots < c_Q$. Block (i, j) has height $h_i = r_{i+1} - r_i$ and width $w_j = c_{j+1} - c_j$. Its column-major values are packed without rounding its dimensions to a global block size, and its contribution is

$$C[r_i : r_{i+1}, :] += A_{ij}B[c_j : c_{j+1}, :].$$

Block-row pointers and column indices locate the blocks. Scalar offsets and value offsets are 64-bit. Block indices remain 32-bit. Host validation checks monotonic offsets, dimensions, index bounds, and payload sizes. A matrix owns its GPU storage and row classifications. Execution plans hold non-owning views, so the matrix must outlive its plans.

3.1 Output ownership and accumulation

For RHS 16, 32, and 64, one cooperative thread array, or CTA, owns one block row. RHS 8 uses one 256-thread CTA per block row when the matrix has at least 512 block rows. Smaller grids use two 256-thread CTAs per block row with disjoint output tiles. Threads outside the actual row height do no output work. Each thread accumulates every block contribution before writing its outputs once. Different CTAs never update the same scalar, which removes the need for output atomics or partial-output reduction storage. Work imbalance between rows can still limit performance.

The tiled RHS-8 path uses two accumulators per thread, and its scalar fallback uses one. RHS 16 and 32 use eight output columns per thread, and RHS 64 uses sixteen. Multiple accumulators let each loaded sparse value feed several independent multiply-add operations, reducing redundant loads at the cost of more registers and fewer independent threads.

Ownership and tile sequence for one block row

Ownership	One CTA writes the entire output panel of block row i .
Initialization	Each thread sets its owned accumulators to zero.
First tile	Load the first reusable B tile into shared memory.
Pipeline step	Initiate the next tile copy into the alternate buffer, then compute the current block contribution.
Boundary	Wait for copies and synchronize the CTA before consuming the next tile or reusing a buffer.
Completion	Write each owned output once, including zero for an empty row.

Figure 1: The tall-row RHS-32 algorithm with its alternating two-buffer schedule. Other widths retain disjoint output ownership, with two output tiles per block row for the small-grid RHS-8 fallback.

3.2 Eight-row tiling for RHS 8

The RHS-8 kernel maps each block row to one 256-thread CTA for matrices with at least 512 block rows. Each warp owns eight local rows and all eight RHS columns. For thread index t , its local row and first RHS column are

$$\ell = 8\lfloor t/32 \rfloor + (t \bmod 8), \quad q = 2(\lfloor t/8 \rfloor \bmod 4).$$

Active threads satisfy $t < 4h_i$ and accumulate columns q and $q + 1$. Since $h_i \leq 64$, at most 256 threads cover the complete output panel. Groups of eight lanes access contiguous local rows, and each sparse value feeds two independent accumulators. The mapping replaces division by the runtime row height with constant index arithmetic. It uses direct dense input loads and no shared memory or CTA barriers. Empty rows write zeros.

The small-grid fallback was selected after an exploratory sweep showed inconsistent gains at 256 block rows. The threshold is empirical and is not claimed optimal between sampled sizes or on other GPUs. Section 5.4 measures its effect against the scalar mapping.

3.3 Staging and dispatch

For RHS 32 and 64, every local output row consumes the same dense input tile for a stored block. Cooperative staging loads this tile once per CTA and reuses it across local rows. A shared leading dimension of 65 floats reduces power-of-two bank-alignment conflicts. The synchronous RHS-32 and RHS-64 kernels use 8,320 and 16,640 shared-memory bytes per CTA, respectively. RHS 16 retains direct loads because its smaller reuse opportunity did not repay the measured synchronization cost.

For RHS 32, rows of at most 16 scalars use a 128-thread single-buffer CTA. Taller rows use a 256-thread CTA with two tiles, consuming 16,640 shared-memory bytes. The next input tile is prefetched before arithmetic on the current tile. Pipeline waits and CTA barriers protect visibility and buffer reuse. A mixed-height matrix is split into two launches only when its mean block degree is at least eight. Homogeneous matrices need only the applicable launch. At lower mean degree, the mixed matrix uses the unsplit policy.

Double buffering is not enabled for RHS 64. A development experiment increased shared allocation to about 32.5 KiB per CTA and regressed a representative median from 2.891 to 4.419 ms. This observation motivated a width-specific policy rather than a universal pipeline. All dispatch thresholds were selected from development sweeps and held fixed for the main library comparison.

4 Experimental design

The main campaign measures the optimized dispatch described above against persistent library controls. All synthetic and application measurements were collected with the current implementation. The separate kernel ablations compare direct implementations under their stated protocols.

4.1 Persistent library controls

Scalar CSR. Synthetic dense blocks are expanded into scalar CSR once. Plans retain arrays, descriptors, handles, and workspace. We measure the default algorithm, explicit CSR algorithm 1 with preprocessing, algorithm 2, and algorithm 3 with preprocessing. All use FP32 compute, column-major panels, and the same non-transpose operation. Preprocessing and initial descriptor creation occur during setup or warmup, outside timing. NVIDIA documents preprocessing benefits for algorithms 1 and 3 [3].

The *CSR lower envelope* is the minimum GPU median among those four configurations within each process. Selecting on measured data favors this comparator. The envelope has no deployed tuning policy or charged tuning cost. It lets us check whether a direct-method win depends on the default library algorithm. Host ratios use a separate minimum over host medians, so they need not select the same algorithm.

Grouped cuBLAS. Blocks with equal height and width are grouped within each ordinal block slot. A slot contains at most one block from each row, so its GEMM outputs do not overlap. Slots execute sequentially. The first uses $\beta = 0$, and subsequent slots use $\beta = 1$. The handle, packed values, shape groups, and device pointer arrays persist.

The cached control refreshes input and output pointer arrays only when their base addresses change. It omits the full output clear when every row is nonempty. For matrices containing empty rows it retains the clear to initialize those rows. A separate changing-address experiment alternates between two allocated input/output pairs and includes pointer refreshes. It is paired with a direct method that alternates the same pairs. Pointer contents are rebuilt on the host and copied on the execution stream when needed. The first measured call reuses the final warmup addresses. The remaining 19 calls alternate addresses and refresh the grouped pointers. Further optimization of pointer marshaling may improve this library composition.

Fixed-block BSR. The uniform-32 synthetic subset is also measured with cuSPARSE BSR algorithm 1. Block values are converted to the required row-major block layout outside timing, while dense panels remain column-major. This gives a format-matched control where variable blocking is unnecessary.

4.2 Synthetic workloads

The core grid contains 128 configurations with 4,096 block rows and columns, degrees 2, 4, 8, 16, four panel widths, four size distributions, and two locality patterns. Uniform uses size 32. Low variance samples $\{24, 32, 40\}$, high variance samples $\{8, 16, \dots, 64\}$, and bimodal samples $\{8, 16, 48, 64\}$. Row and column dimensions are drawn independently. Every row in the core grid has exactly the requested degree. Columns are sampled without replacement and sorted. Random locality spans all block columns. Local locality samples cyclic offsets within plus or minus the requested degree. Stored values are uniform on $[-1, 1]$.

The extension adds 40 configurations. It evaluates 256, 1,024, and 4,096 block rows and columns for high-variance and bimodal sizes, random locality, all panel widths, and either constant degree 16 or the repeating degree pattern 0, 1, 16, 16. The latter has mean degree 8.25 and explicitly includes empty rows. It is constructed by retaining zero blocks, one block, or all blocks from successive rows of the degree-16 parent matrix. Eight constant-degree 4,096-row configurations already belong to the core grid. Thus the union contains 168 unique configurations.

The synthetic campaign holds matrix seed 1 fixed and repeats each configuration in three fresh processes. The dense input contains independent pseudorandom uniform values on $[-1, 1]$, generated with seed 9123. Changing-address buffers contain identical values to isolate address-dependent work.

4.3 Published matrices and partitioning

We also test BCSSTK13, BCSSTK14, and BCSSTK15 from the Harwell-Boeing collection, downloaded through SuiteSparse [10]. They originate from fluid-flow eigenvalues, the Omni Coliseum roof, and an offshore platform module, respectively [11]. Their original orders are 2,003, 1,806, and 3,948.

Symmetric Matrix Market input is expanded, duplicates are combined, and explicit zeros are removed. Values are divided by the maximum absolute entry and converted to FP32. The rows and columns are padded only at the outer boundary to the next multiple of eight. In the original scalar ordering, row partitions cycle through 8, 16, 32, 64 and column partitions through 64, 32, 16, 8. The final partition is truncated to the remaining multiple of eight. A block is stored if it contains a source nonzero. Other entries inside that block are represented as zeros.

This deterministic procedure preserves application-derived values and sparsity after normalization, but uses artificial partitions in place of native finite-element or hierarchical partitions. The measured compact-CSR controls retain only source nonzeros in the padded scalar domain, using explicit algorithms 1, 2, and 3 with preprocessing for 1 and 3. Their lower envelope is computed as above. Cached grouped cuBLAS uses the identical packed blocks as the direct method. This separates the cost of block fill from scalar sparse execution. All four panel widths and three fresh processes are retained. The partitions have 68, 61, and 132 block rows, respectively, so RHS 8 uses the scalar fallback on these application inputs.

4.4 Timing and correctness

Each method receives five warmups and 20 measured executions. Methods are randomly ordered within each process. Configuration and process-label jobs are shuffled before serial execution, with one GPU benchmark active at a time. Synthetic order uses seed 20260905 and application order uses 20260906. Three process labels identify repeated observations of the same matrix, rather than three different matrix instances.

CUDA events bracket each execute call on the default stream, followed by an end-event synchronization. Synchronized host durations are also retained. The intervals measure execution calls, including any device idle time while the host submits later work. This matters particularly for library compositions with multiple calls, so the intervals do not isolate kernel-instruction time. Allocation, format conversion, initial input transfer, and correctness validation are excluded. Every raw GPU and host duration, method position, command, and process status is saved. Medians use the mean of the two middle observations for an even sample count.

The revision uses the RTX 5060 Ti with compute capability 12.0, CUDA runtime and driver API 13.4, Windows, MSVC 19.44, Release compilation, native CUDA architecture, and fast FP32 math. The Windows display driver reports version 32.0.16.1656. The GPU is not clock-locked or isolated from all operating-system activity. Per-process environment records accompany the synthetic timings.

The correctness suite covers both localities for every combination of four size distributions, four widths, and degrees 1, 4, 8, 16, giving 128 cases. Sixteen additional cases contain an empty first row. It tests explicit CSR algorithms, uniform BSR, cached pointer reuse and address changes, and non-default streams. A deterministic test also covers all 64 block shapes in the tiled RHS-8 path, including empty rows and NaN-initialized output to detect missing writes. Full output is compared with a CPU reference that accumulates the entire dot product in double before one final FP32 cast. Failure thresholds are maximum absolute error 5×10^{-4} or relative ℓ_2 error 5×10^{-5} , with NaN and infinity rejected.

Each timed synthetic matrix also receives 64 CPU output probes per method outside timing. Every application output is checked against a separate compact-CSR CPU reference, which also tests the packing transformation. The expanded test executable also passes Compute Sanitizer memcheck with zero reported errors. Racecheck restricted to the project’s row-owned kernels reports zero hazards. Library kernels are outside that racecheck filter. These checks do not substitute for sanitizer coverage of every timed configuration.

4.5 Aggregation and uncertainty

For configuration c , process label p , comparator b , and direct method d , let

$$y_{c,p} = \log(t_{b,c,p}/t_{d,c,p}), \quad g_c = \exp\left(\frac{1}{3} \sum_{p=1}^3 y_{c,p}\right).$$

The reported geometric mean is $G = \exp(|\mathcal{C}|^{-1} \sum_{c \in \mathcal{C}} \log g_c)$. Each configuration and each process within it has equal weight. Ratios above one favor the direct method. Minima and win counts refer to the configuration ratios g_c , not to individual timing trials.

We also report the range of three aggregate ratios obtained by holding the process label fixed. This is a descriptive repeatability check, not a confidence interval or a test of significance. Three process observations provide limited information about tails and shared thermal drift. No population-level confidence interval is assigned to this deliberately chosen grid, and no comparison of interval widths is interpreted as a decomposition of uncertainty. Matrix-instance variation, workload selection, process noise, and hardware variation remain distinct questions.

5 Results

5.1 Core synthetic grid

Table 1 compares execution times on the core grid. The direct path has a geometric speedup of 3.518 against the CSR lower envelope and 4.769 against cached grouped cuBLAS. The respective win counts are 128 and 128 out of 128 configurations. These ratios apply to the tested implementations and execution contract, without establishing a universal advantage over either library. Selecting the CSR algorithm reduces the aggregate ratio from 5.656 for default CSR to 3.518 for the lower envelope. Figure 2 shows the spread behind these averages. Its sorted curves expose the fastest comparators and the width of the performance range across the grid.

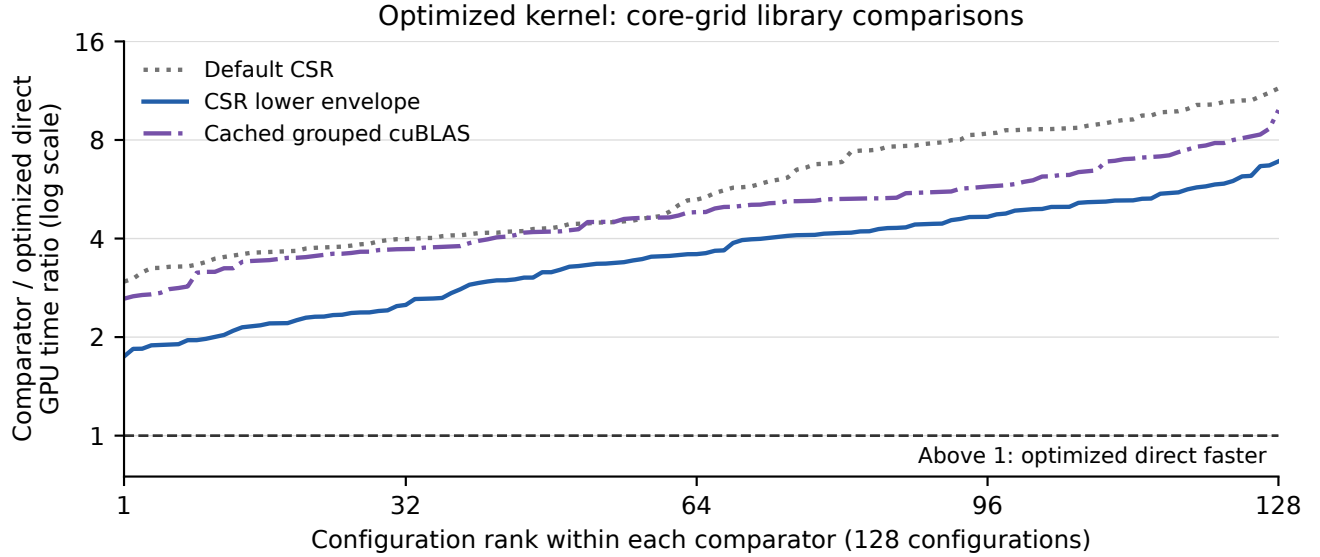


Figure 2: GPU time ratios across the 128 core configurations. Each ratio is the geometric mean of three paired process ratios, computed from per-process timing medians. Each curve sorts configurations independently, so equal ranks need not identify the same matrix. The vertical axis is logarithmic. Ratios above the dashed parity line favor direct execution. These are empirical distributions over the chosen grid, not uncertainty intervals.

Table 1: Core-grid ratios over 128 configurations and three process observations per configuration. Minima and wins use GPU ratios averaged in log space across processes. Changing-address grouped execution is paired with changing-address direct execution.

Comparator	GPU ratio	Host ratio	Minimum	Wins
CSR default	5.656	5.566	2.955	128/128
CSR lower envelope	3.518	3.445	1.745	128/128
Grouped cached	4.769	4.670	2.620	128/128
Grouped changing	4.769	4.708	2.623	128/128

Table 2: Descriptive process-label ranges on the core grid. The two endpoints are observed aggregate ratios, not confidence limits.

Comparator	Geometric mean	Process-label range
CSR default	5.656	5.650 to 5.668
CSR lower envelope	3.518	3.515 to 3.520
Grouped cached	4.769	4.747 to 4.805
Grouped changing	4.769	4.764 to 4.776

Table 3 breaks down the results by width and degree. Each row averages the four size distributions and two localities. Direct time is the geometric mean of the corresponding process medians. Absolute times distinguish small launch-sensitive cases from larger products. The CSR column uses the lower envelope, and grouped execution uses cached pointers.

Table 3: Width and degree breakdown for the core grid. Times are milliseconds and speedups are dimensionless.

RHS	Degree	Direct ms	CSR ratio	Grouped ratio
8	2	0.159	1.907	5.657
8	4	0.277	2.773	6.666
8	8	0.512	4.222	7.725
8	16	0.983	2.158	7.845
16	2	0.234	2.209	3.899
16	4	0.404	2.557	4.669
16	8	0.738	3.444	5.206
16	16	1.404	2.850	5.592
32	2	0.284	3.436	3.449
32	4	0.509	3.650	4.090
32	8	0.836	4.480	5.099
32	16	1.584	5.015	5.529
64	2	0.416	4.614	2.735
64	4	0.729	4.987	3.417
64	8	1.356	5.299	3.831
64	16	2.593	6.007	4.150

5.2 Fixed blocks, scaling, and degree imbalance

Both partitions have constant extent in uniform matrices, making fixed-block BSR a suitable control. Across the 32 uniform configurations, the BSR ratio is 0.834, with 9 direct wins. Table 4 shows the width dependence. Scalar CSR alone would leave this fixed-block comparison untested. BSR wins all eight cases at RHS 8 and all eight at RHS 16. It wins seven of eight cases at RHS 32, while the direct method wins all eight RHS-64 cases. The result favors a fixed-block library path for most of this subset.

Table 4: Fixed-block BSR divided by direct execution on the uniform-32 subset. Each width contains eight degree and locality configurations.

RHS	BSR ratio	Minimum	Wins
8	0.744	0.714	0/8
16	0.587	0.559	0/8
32	0.928	0.904	1/8
64	1.190	1.111	8/8

Table 5 compares constant degree and the imbalanced pattern at three sizes. Each row includes both high-variance and bimodal dimensions and all four panel widths. The workload changes affect both total work and load balance. These results therefore test sensitivity to the combined changes and do not isolate the cost of imbalance at a fixed nonzero count. The tested maximum row degree remains 16, so heavy-tailed application degrees are not covered by this synthetic extension.

Table 5: Size and degree-pattern controls with random column locality. Each row averages eight configurations and three process observations.

Block rows	Degree pattern	CSR ratio	Grouped ratio	CSR wins
256	16 in every row	2.019	4.538	8/8
256	0, 1, 16, 16	1.277	4.065	4/8
1024	16 in every row	2.463	4.482	8/8
1024	0, 1, 16, 16	2.877	4.140	8/8
4096	16 in every row	3.392	5.213	8/8
4096	0, 1, 16, 16	3.963	4.655	8/8

5.3 Application-derived inputs

Table 6 compares the packed direct method with the compact-CSR lower envelope. The aggregate ratio across the twelve matrix and width configurations is 0.296, with 0 direct wins. The block fill column counts source nonzeros divided by stored packed scalars. It shows how much of the packed storage holds source nonzeros. Compact CSR wins every application configuration, with a geometric speedup of 3.382 over the packed direct method. The direct method still beats cached grouped cuBLAS on the same packed inputs, with an aggregate ratio of 6.924. Its descriptive process-label range is 6.020 to 7.851, wider than the corresponding range on the core synthetic grid. The direct kernel executes the packed representation efficiently relative to grouped cuBLAS, while compact CSR remains faster for these inputs.

Table 6: Compact-CSR lower-envelope time divided by packed direct time for published matrices under the stated artificial partitions. A ratio below one favors compact CSR. Each cell averages three process observations.

Matrix	Block fill	RHS 8	RHS 16	RHS 32	RHS 64
bcsstk13	8.9%	0.188	0.129	0.215	0.326
bcsstk14	12.2%	0.352	0.253	0.537	0.599
bcsstk15	6.9%	0.262	0.200	0.413	0.426

These application-derived matrices test the limits of the packed representation. They do not validate the motivating hierarchical use case. Low block fill forces the direct method to execute arithmetic on internal zeros that compact CSR omits. Partition quality and application-native block density must therefore be measured before transferring the synthetic conclusions to a solver.

Figure 3 brings the two format controls together. The uniform subset has one direct win at RHS 32 and eight at RHS 64. Compact CSR remains faster at every tested width for all three artificially partitioned matrices.

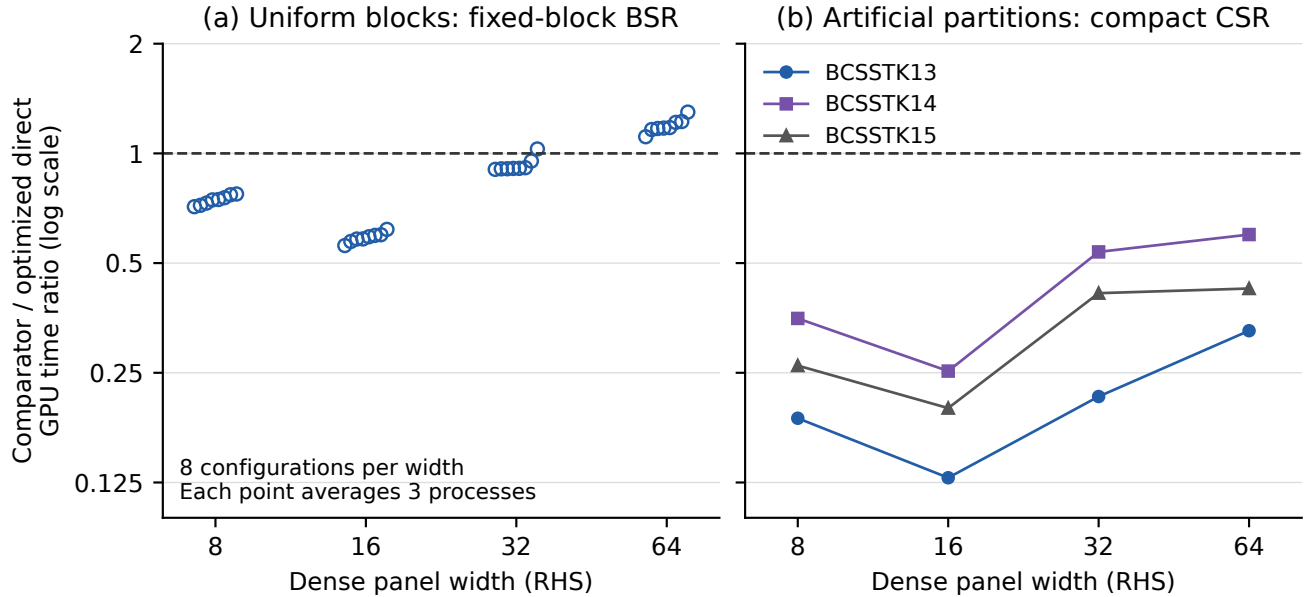


Figure 3: Format controls with a shared logarithmic ratio scale. Each point is the geometric mean of three paired process ratios. Left: all 32 uniform configurations, with eight degree and locality combinations per width. Horizontal offsets separate the points and have no numerical meaning. Right: the compact-CSR lower envelope for the twelve application configurations. Lines connect widths of the same matrix. Ratios below the dashed parity line favor the comparator. The panels represent different workloads and should not be read as a paired comparison between BSR and compact CSR.

5.4 RHS-8 mapping ablation

The RHS-8 ablation covers 144 configurations at 256, 512, 1,024, and 4,096 block rows and columns. At each size, 32 configurations combine degrees 2, 4, 8, 16, the four shape distributions, and both locality patterns. Four additional configurations combine high-variance or bimodal shapes with both localities and repeating degrees 0, 1, 16, 16.

Each configuration uses matrix seeds 1, 2, and 3, with one fresh process per seed. These are different instances, not repeated processes of one fixed instance. The 432 processes retain 25,920 raw timed calls. Each method receives five warmups and 30 measured calls. Method order is randomized within each process, and jobs are shuffled before serial execution. Input generation, CUDA-event placement, and synchronized host timing use the same definitions as the main campaign. For each configuration, we average the three paired log ratios of GPU medians, then give each configuration equal weight. No significance interval is assigned. The comparator is the scalar RHS-8 kernel, and the tiled method executes the current dispatch, including its small-grid fallback.

Table 7: Scalar baseline time divided by optimized direct time for RHS 8. Each size contains 36 configurations and three matrix seeds. The 256-row paths execute the same scalar kernel, so their deviation from parity reflects measurement variation.

Block rows	RHS-8 mapping	Speedup
256	Original scalar	0.995
512	Eight-row tiles	1.278
1,024	Eight-row tiles	1.491
4,096	Eight-row tiles	2.002

Table 7 shows increasing benefit with grid size. All 108 configurations using the tiled path improve after averaging across seeds, with geometric speedup 1.563 and minimum 1.028. Across all 144 configurations, including the unchanged fallback, the ratio is 1.396. The 4,096-row bimodal/random/degree-16 case improves from 2.245 to 1.086 ms, a ratio of 2.067. These absolute times are geometric means of the three seed medians. The minimum is a descriptive observation, not a statistical guarantee against regression.

Every process passes 64 CPU output probes per method on its timed matrix. A deterministic full-output test exercises all 64 supported block shapes in the tiled path, with empty rows, a non-default stream, and output initialized to NaNs to detect missing writes. The updated suite passes memcheck with zero errors and racecheck filtered to the new kernel with zero errors or warnings.

For the representative 4,096-row bimodal/random/degree-16 case, Nsight Compute reports 4,096 CTAs instead of 8,192 and 47 registers per thread instead of 40. Eligible warps per scheduler increase from 0.40 to 0.55, and issued warps per scheduler increase from 0.26 to 0.32. These workload-specific counters support improved scheduling. Instrumented durations do not enter the speedup estimates. The combined experiment does not isolate the contributions of tiling, accumulator reuse, and CTA count. Its paired direct-kernel measurements are separate from the main library comparisons and use three different seeds.

5.5 Historical development evidence

The original five-seed dataset remains available under `data/publication/`. It contains 128 configurations per seed, one benchmark process per configuration and seed, five warmups, and 20 measured executions. Its saved summaries reproduce geometric ratios of 4.988 over default CSR and 4.222 over the original grouped composition. That composition rebuilt pointers and cleared all outputs on every call. The original harness also used constant dense input and fixed method order.

These historical values are separate from the main comparison with the cached baseline. Differences between the experiments cannot isolate the effect of one optimization because the software environment, input values, timing protocol, and comparator configuration also changed. The old per-seed standard errors describe variation in five whole-grid averages. They do not quantify process noise or establish which source accounts for most uncertainty. The original configuration bootstrap is a resampling-sensitivity analysis of a chosen grid, not evidence about a randomly sampled application population.

The kernel policy was developed using separate single-seed sweeps with 1,024 block rows, three warmups, and seven measured executions. Table 8 reports those exploratory comparisons. Each ratio compares the named successive variants on the stated slice, with equal configuration weights.

Table 8: Exploratory development ablations, separate from the main evaluation.

Change	Comparator	Cases	Ratio
Wider accumulation	Four-accumulator direct	128	1.24
Cooperative staging	Wider accumulation	128	1.18
Wider accumulation and staging	Four-accumulator direct	128	1.46
RHS-32 buffering and dispatch	Single-buffer staging	32	1.11

Historical Nsight Compute measurements support the optimization mechanism. An early difficult RHS-64 case had 99.54% achieved occupancy but only 0.48 eligible warps per scheduler. Long-scoreboard waiting accounted for 90.1% of its issue interval. A subsequent staging comparison reduced the reported long-scoreboard share from 78.0% to 37.1%, while barrier waiting reached 36.4%. The tall RHS-32 profile confirmed native asynchronous copies. These observations support reduced redundant loading and a shift toward synchronization costs. They are workload-specific counters, and replay-instrumented durations are excluded from speedup calculations.

6 Scope and limitations

The implementation processes packed blocks directly through width specialization, value reuse, and a staging policy chosen through measurement. The absence of output atomics and execute-time temporary workspace follows from row ownership. It does not imply that matrix storage, classification metadata, or setup is free.

All measurements use one GPU and one local operating environment. Three process observations improve the protocol but cannot characterize long-term drift or hardware variation. The main synthetic campaign fixes one matrix seed. The RHS-8 ablation varies three seeds but does not repeat each seed in multiple processes, so instance variation and process variation are not separately estimated. The historical five-seed study varies instances under a different protocol and cannot be combined with it as additional process repetitions.

The core synthetic rows have constant degree, and the extension adds one particular imbalance pattern. The three published matrices use explicit, artificial partitions and are small relative to the main synthetic grid. They do not substitute for native dense variable blocks from an H^2 application. Setup time, storage totals, and amortization over repeated products are not benchmarked as end-to-end application costs.

The CSR lower envelope favors the comparator through selection on measured medians. The grouped implementation is a concrete slot-based composition, not an exhaustive optimization of grouped GEMM. Compatible research kernels, reordering methods, alternative block ensembles, and additional precisions remain outside the comparison. The results describe the tested regimes. They establish neither state-of-the-art performance nor a new sparse format.

7 Reproducibility

Build with `scripts/build.ps1`. The audit targets are `build/Release/vbsr_audit.exe` and `build/Release/vbsr_application_audit.exe`. Run GPU campaigns serially and use fresh output directories:

```
python scripts/run_revision.py --output data/revision-rerun
python scripts/prepare_application.py
python scripts/run_application.py --output data/application-rerun
python scripts/analyze_revision.py --revision data/revision-rerun '
--application data/application-rerun --output build/rerun-tables
```

The PowerShell continuation character joins the final command, which generates separate tables and figures for the fresh runs. The application preparation script requires NumPy, SciPy, and Requests. Figure generation requires

Matplotlib. Source URLs, archive checksums, binary checksums, partitions, normalization factors, and fill fractions are saved with the downloaded matrices.

`scripts/analyze_revision.py` validates the expected configuration and process coverage, method counts, trial indices, positive finite timings, and process success before generating the result tables and numerical macros. It uses raw medians and paired log ratios as specified above. The synthetic source snapshot and manifest identify the exact measured source even if subsequent manuscript or build-target edits change the working tree. Reproduction should use a fresh output directory when changing code or settings.

For the RHS-8 ablation, run `scripts/benchmark_kernel.py` with `--output` naming a fresh directory. The measured campaign is retained under `data/kernel-rhs8-final-20260906/`, including source checksums, a source snapshot, and raw run archives with per-entry checksums. `scripts/analyze_kernel.py` verifies those checksums, process commands, configuration and seed coverage, trial indices, and finite positive timings before generating Table 7 and its numerical macros. `scripts/build_paper.ps1` validates both campaigns before compiling the PDF. The main synthetic and application records are in `data/optimized-revision/` and `data/application/optimized-results/`. The analyzer defaults to these campaigns. The archived September 5 results remain under their original paths for historical comparison.

`scripts/analyze_publication.ps1` regenerates the historical five-seed statistical summaries from `data/publication/`. The PDF is built from `research/paper.tex`. Validation records, commands, and remaining evidence limits are documented in `research/optimized-validation.md`.

8 Conclusion

VarBlockSpMM implements packed variable-block FP32 SpMM with disjoint output ownership and selective dense-tile reuse. The evaluation uses cached grouped pointers, explicit CSR algorithms, preprocessing, and fixed-block BSR controls. Methods run in randomized order across fresh process launches, and all raw timings are retained. The optimized implementation has core-grid geometric ratios of 3.518 over the CSR lower envelope and 4.769 over cached grouped cuBLAS.

The RHS-8 mapping combines eight-row tiles with two accumulators per thread and retains the scalar kernel for small grids. Its ablation improves all 108 configurations using the tiled path, with geometric speedup 1.563 over the scalar baseline across three matrix seeds.

Representation choice affects the result. The tested variable partitions introduce substantial internal zeros in published scalar sparse matrices. Their comparison with compact CSR must therefore remain separate from results for naturally dense synthetic blocks. Application-native partitions are needed to test whether that storage penalty can be avoided. BSR wins 23 of 32 uniform cases, so the direct method should be evaluated as a specialized variable-block option alongside format-matched alternatives. Tests on additional GPUs and native application partitions are needed to establish where this policy is useful beyond the measured settings.

References

- [1] C. Yang, A. Buluç, and J. D. Owens. Design Principles for Sparse Matrix Multiplication on the GPU. *Euro-Par*, 2018. <https://arxiv.org/abs/1803.08601>
- [2] P. Okanovic et al. High Performance Unstructured SpMM Computation Using Tensor Cores. 2024. <https://arxiv.org/abs/2408.11551>
- [3] NVIDIA. cuSPARSE API reference, SpMM algorithms and preprocessing. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cusparse/index.html#cusparsespmmm>
- [4] NVIDIA. cuBLAS grouped batched GEMM. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cublas/#cublas-t-gemmgrouppedbatched>
- [5] W. Boukaram, Y. Liu, P. Ghysels, and X. S. Li. Adaptive Sketching Based Construction of H^2 Matrices on GPUs. *IPDPSW*, 2025. <https://arxiv.org/abs/2506.16759>

- [6] R. Vuduc and H.-J. Moon. Fast Sparse Matrix-Vector Multiplication by Exploiting Variable Block Structure. *HPCC*, 2005. https://doi.org/10.1007/11557654_91
- [7] Berkeley Benchmarking and Optimization Group. OSKI Variable Block Row Format. 2007. https://bebop.cs.berkeley.edu/oski/html/group__MATTYPE__VBR.html
- [8] G. Huang, G. Dai, Y. Wang, and H. Yang. GE-SpMM: General-Purpose Sparse Matrix-Matrix Multiplication on GPUs for Graph Neural Networks. *SC20*, 2020. <https://doi.org/10.1109/SC41405.2020.00076>
- [9] NVIDIA. Asynchronous Data Copies. CUDA Programming Guide. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html>
- [10] SuiteSparse Matrix Collection. HB/BCSSTK13, HB/BCSSTK14, and HB/BCSSTK15. Accessed 5 September 2026. <https://sparse.tamu.edu/HB/bcsstk13>
- [11] NIST Matrix Market. BCS Structural Engineering Matrices. BCSSTK13, BCSSTK14, and BCSSTK15 problem descriptions. <https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/bcsstruc1/bcsstk13.html> <https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/bcsstruc2/bcsstk14.html> <https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/bcsstruc2/bcsstk15.html>